

OtakuPulse Companion — Swipe-App mit Partys (Android + Backend)

Context

Patrick fällt die Auswahl schwer, welchen Anime er als nächstes schauen soll. Er sucht nach Kategorie, Tags (Magie, Isekai, Ecchi, Medical ...) und „ähnlich wie X“, verliert aber den Überblick über Gesehenes und über neue Folgen laufender Simulcasts. Dieselbe Entscheidungsschwäche hat er im Freundeskreis — deshalb soll die App **zu mehreren** benutzbar sein.

Die Web-Seite OtakuPulse (/mnt/cache/appdata/otakupulse) deckt Discovery ab, hat aber bewusst keine Nutzer-Features (Projektnotiz: „Watchlist kommt NUR in der Android-App, nicht Web.“). Diese App ist genau das — plus Party-Funktion.

Kern der Bedienung ist ein Tinder-artiger Kartenstapel: nach links wegswischen = kein Interesse, nach rechts = auf die Watchlist, nach oben = Super-Swipe, der allen in der Party eine Push-Benachrichtigung mit dem Anime schickt. Wischen zwei Party-Mitglieder denselben Titel nach rechts, entsteht ein **Match** — die gemeinsame Party-Liste füllt sich also von allein mit „das wollen beide sehen“.

Getroffene Entscheidungen

Frage	Entscheidung	Warum
Plattform	Native (Kotlin/Compose) Android	Echte Push, Offline, Toolchain steht schon (WorkTracker)
Backend	Eigener otakupulse-companion Container	Rührt die SEO-Seite und deren proxy.ts-Absicherung nicht an
Adresse	app.otakupulse.de, öffentlich im Internet	Freunde müssen von überall reinkommen — kein Tailscale, kein VPN, kein LAN-Zwang
Anime-Daten	OtakuPulse-Postgres, AniList als Fallback	Deutsche Texte, Ger-Dub, Streaming-Anbieter — ~11.000 Titel liegen schon da
Anmeldung	Nur Party-Code, kein Konto	Geschlossener Freundeskreis, nichts zu verwalten
Push	Firebase Cloud Messaging	Einziger Weg für sofortige Zustellung bei geschlossener App
Sicherung	Android Auto Backup + JSON-Export/Import	Restauriert auch das Geräte-Token — ohne das wäre „nur Party-Code“ bei Handywechsel fatal

Verworfen: Der geplante separate „Würfeln“-Screen entfällt — der Swipe-Stapel ist die Entscheidungshilfe, die Filter bestimmen nur noch, was im Stapel liegt.

Voraussetzungen

- Android-Toolchain: source /mnt/cache/appdata/android-build/env.sh, dann "\$GRADLE" :app:assembleDebug (JDK 17, SDK android-35, build-tools 35.0.0, Gradle 8.11.1)
- GitHub-SSH als ppschauss funktioniert (verifiziert); gh CLI fehlt → Schritt 0
- otakupulse-db (postgres:16-alpine) hängt im Docker-Netz **otakupulse-net** → dort

erreichbar als otakupulse-db:5432. Zugangsdaten in
/mnt/cache/appdata/otakupulse/secrets.env

- Unraid hat **kein** compose-Plugin → manage.sh mit docker run, Vorbild
/mnt/cache/appdata/habitbot/manage.sh
- Kein Emulator → nur Build + JVM-Unit-Tests; App wird auf echter Hardware getestet

Schritt 0 — Werkzeug & Repos

1. gh CLI installieren (Release-Tarball nach /usr/local/bin, Unraid hat kein apt).
 2. **Patrick führt aus:** ! gh auth login (interaktiv).
 3. Ein Repo für beides: /root/otakupulse-app mit android/ und server/. git init, .gitignore (Android-Standard + local.properties, keystore.properties, *.jks, secrets.env, google-services.json), README.
 4. gh repo create ppschauss/otakupulse-app --private --source . --remote origin, erster Commit + Push.
 5. Design-Spec nach docs/superpowers/specs/2026-07-21-otakupulse-companion-design.md — Inhalt dieses Plans, ausformuliert — committen.
 6. **Patrick legt an:** Firebase-Projekt + google-services.json (App) und einen Service-Account-Schlüssel (Backend). Beides gitignored.
-

Teil A — Backend (app.otakupulse.de)

Ort: /mnt/cache/appdata/otakupulse-companion/, Code im Repo unter server/. Aufbau nach dem Homelab-Muster: manage.sh (build/up/rebuild/down/restart/logs), secrets.env, build/.

Stack: Python + FastAPI + SQLAlchemy, psycpg. Zwei Datenbank-Verbindungen: **lesend** auf die OtakuPulse-DB (nur SELECT, eigener Postgres-Rolle mit reinem Leserecht) und **schreibend** auf eine eigene Datenbank companion im selben Postgres-Container. Container hängt in otakupulse-net.

Öffentlich erreichbar — ohne Tailscale

app.otakupulse.de läuft über den **bestehenden Cloudflare-Tunnel** auf NPM und von dort in den Container. Damit erreichen deine Freunde die App aus jedem Mobilfunknetz, ohne VPN, ohne Tailscale, ohne im heimischen WLAN zu sein. HTTPS und DDoS-Schutz kommen von Cloudflare, es muss **kein Port am Router geöffnet** werden.

Weil es öffentlich steht, wird nur das Nötigste exponiert: - Jeder Endpunkt außer POST /v1/devices und /health verlangt ein gültiges Geräte-Token - Rate-Limit pro Token **und** pro IP; die Geräte-Registrierung zusätzlich streng gedrosselt, damit niemand Tokens auf Vorrat erzeugt - Kein Admin-Bereich, keine Schreibrechte auf die OtakuPulse-Daten, keine Auflistung fremder Partys - Die Deck-Antworten enthalten nur, was auch auf www.otakupulse.de öffentlich steht — die App wird damit kein bequemer Abzug deiner SEO-Datenbank (Seitengröße gedeckelt) - Getrennter Cloudflare-Hostname: die Absicherung der SEO-Seite in src/proxy.ts bleibt unangetastet

Eigene Tabellen

Tabelle	Inhalt
device	id, token (geheim, Client-Identität), display_name, fcm_token, last_seen
party	id, name, join_code, created_by, created_at
party_member	party_id, device_id, joined_at
swipe	device_id, anime_id, Richtung (LEFT/RIGHT/SUPER), created_at — eindeutig je Gerät+Anime
match	party_id, anime_id, created_at — entsteht, sobald zwei Mitglieder RIGHT gewischt haben
anime_cache	Von AniList nachgeladene Titel, die in der OtakuPulse-DB fehlen

Endpunkte

- POST /v1/devices — registriert ein Gerät, gibt token zurück; PATCH aktualisiert Name und FCM-Token
- POST /v1/parties / POST /v1/parties/join (per join_code) / GET /v1/parties/{id} (Mitglieder, Matches)
- GET /v1/deck — liefert den Swipe-Stapel: Filter als Query-Parameter (Genres, Tags, Jahr, Format, Ger-Dub, Anbieter), sortiert nach Score, **bereits gewischte Titel ausgeschlossen**, seitenweise ~20 Karten
- POST /v1/swipes — nimmt einen Swipe entgegen; bei RIGHT wird auf Matches geprüft, bei SUPER geht sofort Push an alle anderen Party-Mitglieder
- GET /v1/anime/{id} — Detail inkl. „Ähnliche“ (aus OtakuPulse-Relationen, Fallback AniList-recommendations)
- GET /v1/airing — Airing-Plan für eine Liste von IDs (aus AniList, serverseitig gecacht)

Authentifizierung: Authorization: Bearer <device-token>. Kein Konto, kein Passwort. Ein Gerät ohne gültiges Token bekommt 401. Zusätzlich einfaches Rate-Limit je Token, damit ein verlorenes Token keinen Schaden anrichtet.

AniList-Fallback: Fehlt eine ID in der OtakuPulse-DB, holt das Backend sie live von <https://graphql.anilist.co> und legt sie in anime_cache ab. Wichtig: AniList lässt nur ~30 **Anfragen/Minute** zu und antwortet sonst mit 429 — das Backend drosselt zentral (Token-Bucket) und respektiert Retry-After. Genau daran ist beim OtakuPulse-Importer schon mal etwas gescheitert; und ein --only-missing-Blast gegen JustWatch hat dort seinerzeit eine 403-Sperre ausgelöst. Also bewusst langsam.

Push: FCM-HTTP-v1-API mit Service-Account-Schlüssel. Beim Super-Swipe erhält jedes andere Party-Mitglied eine Nachricht mit Titel, Cover und Anime-ID; tote FCM-Tokens werden dabei aufgeräumt.

Teil B — Android-App

Struktur analog /root/worktracker, Package de.pattaku.otakupulse.app, minSdk 26 / targetSdk 35, Compose M3, Room, manuelle DI (AppContainer), Version-Catalog. Neu: Retrofit/Ktor + kotlinx-serialization, coil-compose, androidx-work-runtime-ktx, firebase-messaging, navigation-compose.

data/api/	CompanionClient (Interface) + Implementierung, DTOs
data/local/	Room: Deck-Cache, watchlist_entry, episode_seen, ausstehende Swipes
data/repo/	DeckRepository, WatchlistRepository, PartyRepository
domain/	Anime, WatchStatus, SwipeDirection, DeckFilter
di/	AppContainer
ui/swipe/	Kartenstapel — Hauptbildschirm
ui/watchlist/	Statuslisten, Fortschritt, Badge bei neuer Folge
ui/calendar/	Simulcast-Wochenansicht
ui/party/	Party erstellen/beitreten, Mitglieder, Matches
ui/detail/	Anime-Detail inkl. „Ähnliche“
ui/theme/	Material3, Pink #ff4d6d + Violett #8b5cf6 (wie OtakuPulse)
work/	EpisodeCheckWorker, SwipeSyncWorker
push/	FirebaseMessagingService
backup/	JSON-Export/Import via SAF

Der Kartenstapel

Eigene Compose-Implementierung (pointerInput + detectDragGestures, Karte folgt dem Finger mit Rotation, farbige Überlagerung je Richtung, Schwellenwert löst aus). Keine Fremdbibliothek — das ist überschaubarer Code und du bist nicht von einem ungepflegten Paket abhängig.

- **Links** → verworfen
- **Rechts** → Watchlist (Status PLANNED)
- **Oben** → Super-Swipe: Push an alle in der Party, landet zusätzlich auf der eigenen Watchlist
- **Tippen** → Detailansicht

Der Stapel wird vorausgeladen: sind weniger als fünf Karten übrig, holt die App die nächste Seite. Swipes werden lokal gespeichert und von einem `SwipeSyncWorker` zum Backend geschoben — so funktioniert Wischen auch ohne Netz.

Neue Folgen

`EpisodeCheckWorker` (`WorkManager`, ~alle 4 h, nur bei Netz) fragt `/v1/airing` für die Anime mit Status `WATCHING` ab und markiert neu erschienene Folgen. Daraus entsteht beides: die lokale Benachrichtigung **und** das Badge in der Watchlist — eine Quelle, zwei Anzeigen.

Ab Android 13 braucht das die `POST_NOTIFICATIONS`-Runtime-Berechtigung; sie wird beim ersten Party-Beitritt erfragt, nicht beim Start.

Sicherung

`android:allowBackup="true"` + Backup-Regeln → Room-DB **und Geräte-Token** landen in Google Drive und kommen bei Neuinstallation zurück. Zusätzlich manueller JSON-Export/-Import über den System-Dateidialog, Vorbild ist das export-Package in `/root/worktracker`. Import ist ein Merge, kein Überschreiben.

Fehlerfälle

- **Kein Netz** → Deck-Cache anzeigen, Swipes lokal puffern, dezenter Offline-Hinweis
- **Backend nicht erreichbar** → App bleibt bedienbar (Watchlist ist lokal), Party-Ansicht zeigt letzten bekannten Stand
- **AniList 429** → Backend drosselt, App merkt nichts solange der Cache reicht
- **Stapel leer** → Leerzustand mit Vorschlag, Filter zu lockern
- **Ungültiges Token** (401) → Hinweis mit Angebot, aus einem Export wiederherzustellen
- **Worker-Fehler** → `Result.retry()`, keine Benachrichtigung

Tests

Backend (pytest): Deck-Auswahl schließt gewischte Titel aus und respektiert Filter · Match entsteht genau beim zweiten RIGHT · Super-Swipe erreicht alle außer dem Absender · AniList-Fallback nur bei fehlender ID, mit Drosselung · Party-Beitritt mit falschem Code scheitert.

Android (JVM, kein Emulator): DTO-Parsing gegen echte Backend-Antworten als Fixtures · Swipe-Puffer-Synchronisation inklusive Wiederholung nach Fehler · „Neue Folge“-Diff-Logik · Export→Import-Rundlauf ergibt identischen Zustand · Deck-Nachladelogik. `CompanionClient` steckt hinter einem Interface und ist damit fakebar.

Reihenfolge

Meilenstein	Inhalt
M1	Repo + gh, Backend-Gerüst mit <code>manage.sh</code> , Lese-Rolle auf <code>OtakuDance-DB</code> , <code>/v1/deck</code> liefert echte Titel
M2	Android-Gerüst, Kartenstapel gegen <code>/v1/deck</code> , Detailansicht
M3	Geräte-Registrierung, Swipes, lokale Watchlist, Offline-Puffer
M4	Partys, Matches, Super-Swipe + FCM
M5	Kalender + <code>EpisodeCheckWorker</code> + Badge
M6	Auto Backup + JSON-Export/Import, <code>app.otakupulse.de</code> über NPM/Tunnel veröffentlichen, signierte APK nach dist/

Jeder Meilenstein wird einzeln committet und gepusht.

Verifikation

1. **Backend baut und läuft:** ./manage.sh rebuild && ./manage.sh logs — Container gesund, curl localhost:<port>/health = 200.
 2. **Backend-Tests:** docker exec otakupulse-companion pytest — Ausgabe zeigen, nicht behaupten.
 3. **Echte Daten:** curl gegen /v1/deck mit Tag-Filter liefert deutsche Beschreibungen aus der OtakuPulse-DB; einmal mit einer bewusst unbekannten AniList-ID den Fallback auslösen.
 4. **Android baut:** source /mnt/cache/appdata/android-build/env.sh && "\$GRADLE" :app:assembleDebug sowie :app:testDebugUnitTest.
 5. **Auf echter Hardware:** APK installieren. Prüfen: Stapel lädt → links/rechts wischen → Titel steht auf der Watchlist → Party erstellen, zweites Gerät tritt per Code bei → beide wischen denselben Titel rechts = Match erscheint → Super-Swipe erzeugt auf dem anderen Gerät eine Push-Nachricht → Kalender zeigt die laufende Season → Export schreiben, App-Daten löschen, Import stellt Watchlist **und** Party-Mitgliedschaft wieder her.
 6. **Von unterwegs, ohne VPN:** Handy im Mobilfunknetz, Tailscale **aus**, nicht im Heim-WLAN → <https://app.otakupulse.de/health> liefert 200 und die App funktioniert vollständig. Gegenprobe: Aufruf ohne Token gibt 401.
 7. **SEO-Seite unversehrt:** <https://www.otakupulse.de/legal/admin> und [/api/users/login](https://www.otakupulse.de/api/users/login) liefern weiterhin 404, / liefert 200 — das Backend darf daran nichts geändert haben.
- Stolperstein aus der Vergangenheit:** WorkTracker ließ sich auf MIUI/HyperOS zunächst nicht installieren („App wurde nicht installiert“) — ein Geräte-Neustart half, nicht Signatur oder Download.